

SIMATS ENGINEERING
CO1 -ASSESSMENT TOOL 1

NAME:K.Harsha Vardhan

Reg.no: 192425228

Experiment 1: Installation of Reinforcement Learning Development

Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

CODE :

```
import gymnasium as gym
env = gym.make("CartPole-v1")
observation, info = env.reset()
print("Initial Observation:")
print(observation)
total_reward = 0
for step in range(5):
    action = env.action_space.sample()
    observation, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    print("\nStep:", step + 1)
    print("Action:", action)
    print("Observation:", observation)
    print("Reward:", reward)
    if terminated or truncated:
        print("Episode Ended!")
        break
env.close()
print("\nTotal Reward:", total_reward)
```

OUTPUT:

```
Initial Observation:
[-0.01301366 -0.01999307  0.0032603 -0.0462355 ]

Step: 1
Action: 0
Observation: [-0.01341352 -0.21516162  0.00233559  0.2474743 ]
Reward: 1.0

Step: 2
Action: 0
Observation: [-0.01771675 -0.41031685  0.00728508  0.540893 ]
Reward: 1.0

Step: 3
Action: 1
Observation: [-0.02592309 -0.21529806  0.01810294  0.25051436]
Reward: 1.0

Step: 4
Action: 0
Observation: [-0.03022905 -0.41067377  0.02311322  0.5488519 ]
Reward: 1.0

Step: 5
Action: 1
Observation: [-0.03844253 -0.215884  0.03409026  0.26353997]
Reward: 1.0

Total Reward: 5.0
```

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

CODE :

```
import gymnasium as gym
environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]
for env_name in environments:
    print("\n=====")
    print("Environment:", env_name)
    print("=====")
    env = gym.make(env_name)
    observation, info = env.reset()
    print("Initial State:", observation)
    total_reward = 0
    for step in range(3):
        action = env.action_space.sample()
        observation, reward, terminated, truncated, info = env.step(action)
        print("\nStep:", step + 1)
        print("Action:", action)
        print("Next State:", observation)
        print("Reward:", reward)
        total_reward += reward
        if terminated or truncated:
            print("Episode Finished")
            break
    print("Total Reward:", total_reward)
env.close()
```

OUTPUT:

```
=====
Environment: FrozenLake-v1
=====
Initial State: 0

Step: 1
Action: 1
Next State: 0
Reward: 0

Step: 2
Action: 3
Next State: 0
Reward: 0

Step: 3
Action: 2
Next State: 0
Reward: 0
Total Reward: 0

=====
Environment: CartPole-v1
=====
Initial State: [ 0.02373488 -0.00461538  0.02087828 -0.02409822]

Step: 1
Action: 0
Next State: [ 0.02364257 -0.20003043  0.02039632  0.27509823]
Reward: 1.0

Step: 2
Action: 1
Next State: [ 0.01964196 -0.00520533  0.02589828 -0.01108253]
Reward: 1.0

Step: 3
Action: 0
Next State: [ 0.01953786 -0.20068894  0.02567663  0.2896578 ]
Reward: 1.0
Total Reward: 3.0

=====
Environment: MountainCar-v0
=====
Initial State: [-0.43737996  0.          ]

Step: 1
Action: 1
Next State: [-0.43801942 -0.00063945]
Reward: -1.0

Step: 2
Action: 1
Next State: [-0.43929368 -0.00127427]
Reward: -1.0

Step: 3
Action: 0
Next State: [-0.44219354 -0.00289984]
Reward: -1.0
Total Reward: -3.0
```

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

CODE :

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", is_slippery=False)
state, info = env.reset()

print("Initial State:", state)

total_reward = 0
for step in range(7):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)

    print("\nStep:", step + 1)
    print("Current State:", state)
    print("Action Taken:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    total_reward += reward
    state = next_state

    if terminated or truncated:
        print("\nEpisode Finished")
```

break

env.close()

print("\nTotal Reward:", total_reward)

OUTPUT ;

```
Step: 1
Current State: 0
Action Taken: 0
Next State: 0
Reward: 0

Step: 2
Current State: 0
Action Taken: 0
Next State: 0
Reward: 0

Step: 3
Current State: 0
Action Taken: 1
Next State: 4
Reward: 0

Step: 4
Current State: 4
Action Taken: 1
Next State: 8
Reward: 0

Step: 5
Current State: 8
Action Taken: 2
Next State: 9
Reward: 0

Step: 6
Current State: 9
Action Taken: 2
Next State: 10
Reward: 0

Step: 7
Current State: 10
Action Taken: 2
Next State: 11
Reward: 0

Episode Finished
Total Reward: 0
```

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

CODE :

```
states = ["Start", "Middle", "Goal"]
actions = ["Move"]
transition = {
    "Start": "Middle",
    "Middle": "Goal",
    "Goal": "Goal"
}
reward = {
    "Start": 0,
    "Middle": 5,
    "Goal": 10
}
policy = {
    "Start": "Move",
    "Middle": "Move",
    "Goal": "Move"
}
state = "Start"
total_reward = 0

print("===== Markov Decision Process Simulation =====")
```

```

while state != "Goal":

    print("\nCurrent State :", state)

    action = policy[state]
    print("Action Taken :", action)
    next_state = transition[state]
    print("Next State  :", next_state)

    r = reward[next_state]
    print("Reward      :", r)

    total_reward += r
    state = next_state

print("\nGoal Reached Successfully!")
print("Final State :", state)
print("Total Reward:", total_reward)

```

OUTPUT :

```

===== Markov Decision Process Simulation =====

Current State : Start
Action Taken  : Move
Next State    : Middle
Reward        : 5

Current State : Middle
Action Taken  : Move
Next State    : Goal
Reward        : 10

Goal Reached Successfully!
Final State   : Goal
Total Reward: 15

```

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for

calculating state-value and action-value functions and analyze the convergence of value estimation.

CODE:

Bellman Expectation and Bellman Optimality Equation

```
gamma = 0.9 # Discount factor
```

```
print("Bellman Equation Demonstration")
```

```
print("-----")
```

```
# Current state value
```

```
V_s = 10
```

```
# Rewards and next state values
```

```
reward1 = 5
```

```
next_state1 = 8
```

```
reward2 = 2
```

```
next_state2 = 12
```

```
# -----
```

```
# Bellman Expectation Equation
```

```
# -----
```

```
prob1 = 0.5
```

```
prob2 = 0.5
```

```
expectation_value = (
```

```
    prob1 * (reward1 + gamma * next_state1)
```

```
    + prob2 * (reward2 + gamma * next_state2)
```

)

```
print("\nBellman Expectation Value")
print("State Value =", expectation_value)

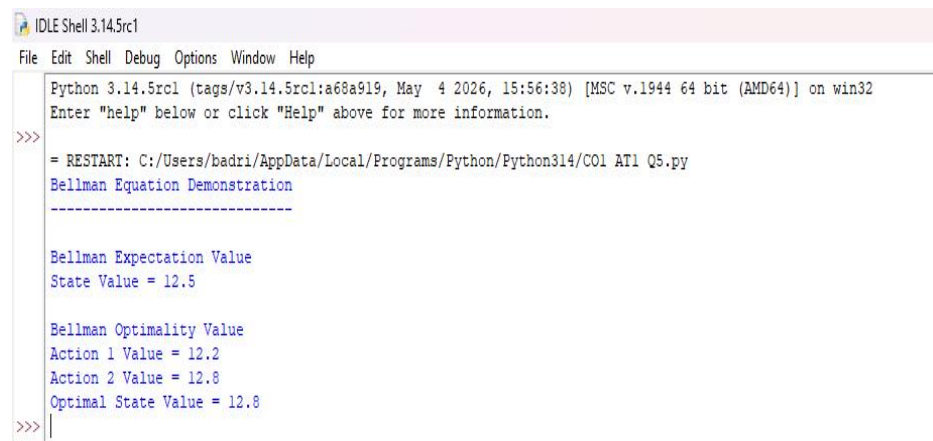
# -----
# Bellman Optimality Equation
# -----

action1 = reward1 + gamma * next_state1
action2 = reward2 + gamma * next_state2

optimal_value = max(action1, action2)

print("\nBellman Optimality Value")
print("Action 1 Value =", action1)
print("Action 2 Value =", action2)
print("Optimal State Value =", optimal_value)
```

OUTPUT:



```
IDLE Shell 3.14.5rc1
File Edit Shell Debug Options Window Help
Python 3.14.5rc1 (tags/v3.14.5rc1:a68a919, May 4 2026, 15:56:38) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/badri/AppData/Local/Programs/Python/Python314/CO1 AT1 Q5.py
Bellman Equation Demonstration
-----

Bellman Expectation Value
State Value = 12.5

Bellman Optimality Value
Action 1 Value = 12.2
Action 2 Value = 12.8
Optimal State Value = 12.8
>>> |
```

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during

Reinforcement Learning.

CODE:import random

Rewards (Q-values) for 5 actions

q_values = [10, 25, 15, 30, 20]

epsilon = 0.2 # 20% exploration

print("Q-Values:", q_values)

Greedy Strategy

print("\nGreedy Strategy")

greedy_action = q_values.index(max(q_values))

print("Selected Action:", greedy_action)

print("Reward:", q_values[greedy_action])

Random Strategy

print("\nRandom Strategy")

random_action = random.randint(0, len(q_values) - 1)

print("Selected Action:", random_action)

print("Reward:", q_values[random_action])

Epsilon-Greedy Strategy

print("\nEpsilon-Greedy Strategy")

if random.random() < epsilon:

Exploration

```

        action = random.randint(0, len(q_values) - 1)
        print("Exploration")
    else:
        # Exploitation
        action = q_values.index(max(q_values))
        print("Exploitation")

print("Selected Action:", action)
print("Reward:", q_values[action])

```

OUTPUT:



```

IDLE Shell 3.14.5rc1
File Edit Shell Debug Options Window Help
Python 3.14.5rc1 (tags/v3.14.5rc1:a68a919, May 4 2026, 15:56:38) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/badri/AppData/Local/Programs/Python/Python314/CO1 AT1 Q6.py
Q-Values: [10, 25, 15, 30, 20]

Greedy Strategy
Selected Action: 3
Reward: 30

Random Strategy
Selected Action: 1
Reward: 25

Epsilon-Greedy Strategy
Exploitation
Selected Action: 3
Reward: 30
>>>

```

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

CODE:

```

import random
import math

true_rewards = [1.0, 2.0, 1.5, 2.5]
n_arms = len(true_rewards)
iterations = 100
epsilon = 0.1

```

```

q_values = [0.0] * n_arms
counts = [0] * n_arms
total_reward = 0

print("Epsilon-Greedy Algorithm")

for i in range(iterations):
    if random.random() < epsilon:
        action = random.randint(0, n_arms - 1)
    else:
        action = q_values.index(max(q_values))

    reward = true_rewards[action] + random.uniform(-0.5, 0.5)

    counts[action] += 1
    q_values[action] += (reward - q_values[action]) / counts[action]
    total_reward += reward

print("Estimated Values:", [round(x, 2) for x in q_values])
print("Action Counts:", counts)
print("Cumulative Reward:", round(total_reward, 2))

print("\nUpper Confidence Bound (UCB) Algorithm")

q_values = [0.0] * n_arms
counts = [0] * n_arms
total_reward = 0

for i in range(n_arms):
    reward = true_rewards[i] + random.uniform(-0.5, 0.5)
    counts[i] = 1
    q_values[i] = reward
    total_reward += reward

```

```

for t in range(n_arms, iterations):
    ucb_values = []

    for i in range(n_arms):
        bonus = math.sqrt((2 * math.log(t + 1)) / counts[i])
        ucb = q_values[i] + bonus
        ucb_values.append(ucb)

    action = ucb_values.index(max(ucb_values))

    reward = true_rewards[action] + random.uniform(-0.5, 0.5)

    counts[action] += 1
    q_values[action] += (reward - q_values[action]) / counts[action]
    total_reward += reward

print("Estimated Values:", [round(x, 2) for x in q_values])
print("Action Counts:", counts)
print("Cumulative Reward:", round(total_reward, 2))

```

OUTPUT:

```

'''===== RESTART: C:/Users/badri/AppData/Local/Programs/Python/Python314/CO1 AT1 Q7.py =====
Epsilon-Greedy Algorithm
Estimated Values: [1.08, 0.0, 1.59, 2.56]
Action Counts: [6, 0, 4, 90]
Cumulative Reward: 242.81

Upper Confidence Bound (UCB) Algorithm
Estimated Values: [1.08, 2.03, 1.61, 2.49]
Action Counts: [4, 14, 7, 75]
Cumulative Reward: 231.06
>>>
'''

```

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

CODE:

```
import random
import matplotlib.pyplot as plt

episodes = 20

episode_rewards = []
cumulative_rewards = []

total_reward = 0

for episode in range(1, episodes + 1):
    reward = random.randint(1, 10)
    episode_rewards.append(reward)

    total_reward += reward
    cumulative_rewards.append(total_reward)

print("Episode Rewards:")
print(episode_rewards)

print("\nCumulative Rewards:")
print(cumulative_rewards)

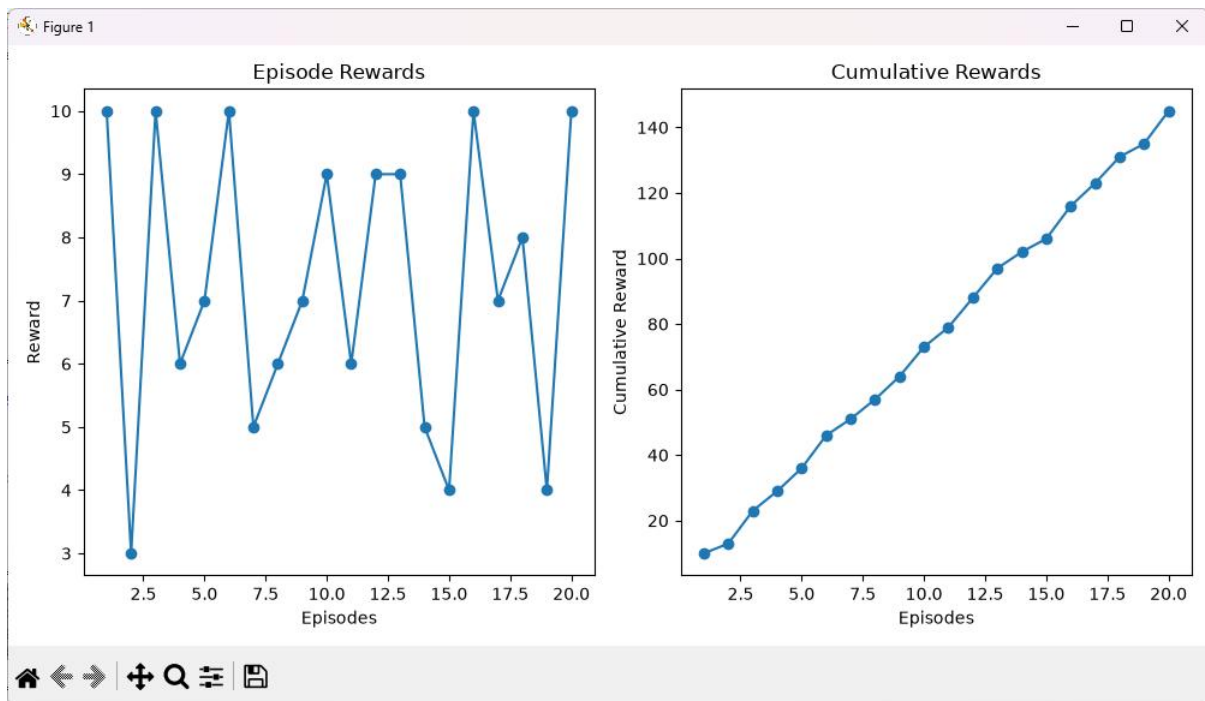
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, episodes + 1), episode_rewards, marker='o')
plt.title("Episode Rewards")
plt.xlabel("Episodes")
plt.ylabel("Reward")
```

```
plt.subplot(1, 2, 2)
plt.plot(range(1, episodes + 1), cumulative_rewards, marker='o')
plt.title("Cumulative Rewards")
plt.xlabel("Episodes")
plt.ylabel("Cumulative Reward")

plt.tight_layout()
plt.show()
```

OUTPUT:



Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

CODE:

```
import tensorflow as tf
from tensorflow import keras
```



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

model = Sequential()

model.add(Dense(24, input_shape=(4,), activation="relu"))
model.add(Dense(24, activation="relu"))
model.add(Dense(2, activation="linear"))

model.compile(optimizer="adam", loss="mse", metrics=["accuracy"])

x_train = np.random.rand(100, 4)
y_train = np.random.rand(100, 2)

model.fit(x_train, y_train, epochs=10, batch_size=10)

print("\nModel Summary")
model.summary()

test = np.array([[0.5, 0.2, 0.1, 0.7]])

prediction = model.predict(test)

print("\nPredicted Value:")
print(prediction)

```

OUTPUT:

```

'''
===== RESTART: C:/Users/badri/AppData/Local/Programs/Python/Python314/COL AT1 Q8.py =====
Episode Rewards:
[5, 4, 7, 6, 4, 4, 4, 5, 5, 9, 7, 3, 3, 1, 9, 9, 10, 10, 7, 8]

Cumulative Rewards:
[5, 9, 16, 22, 26, 30, 34, 39, 44, 53, 60, 63, 66, 67, 76, 85, 95, 105, 112, 120]
'''

```

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

CODE:

```
import gymnasium as gym
import random

env = gym.make("CartPole-v1")

episodes = 10
success = 0

for episode in range(episodes):

    observation, info = env.reset()

    total_reward = 0

    while True:

        action = env.action_space.sample()

        observation, reward, terminated, truncated, info = env.step(action)

        total_reward += reward

        if terminated or truncated:
            break

    print("Episode:", episode + 1)
    print("Reward:", total_reward)
```

```
if total_reward >= 195:
```

```
    success += 1
```

```
env.close()
```

```
print("\nTotal Episodes:", episodes)
```

```
print("Successful Episodes:", success)
```

```
print("Success Rate:", (success / episodes) * 100, "%")
```

OUTPUT:

```
===== RESTART: C:/Users/badri/AppData/Local/Programs/Python/Python314/COL AT1 Q10.py =====
Episode: 1
Reward: 13.0
Episode: 2
Reward: 18.0
Episode: 3
Reward: 17.0
Episode: 4
Reward: 17.0
Episode: 5
Reward: 25.0
Episode: 6
Reward: 45.0
Episode: 7
Reward: 17.0
Episode: 8
Reward: 14.0
Episode: 9
Reward: 19.0
Episode: 10
Reward: 41.0

Total Episodes: 10
Successful Episodes: 0
Success Rate: 0.0 %
>>> |
```